

# SmartJourney: System Overview and Deployment

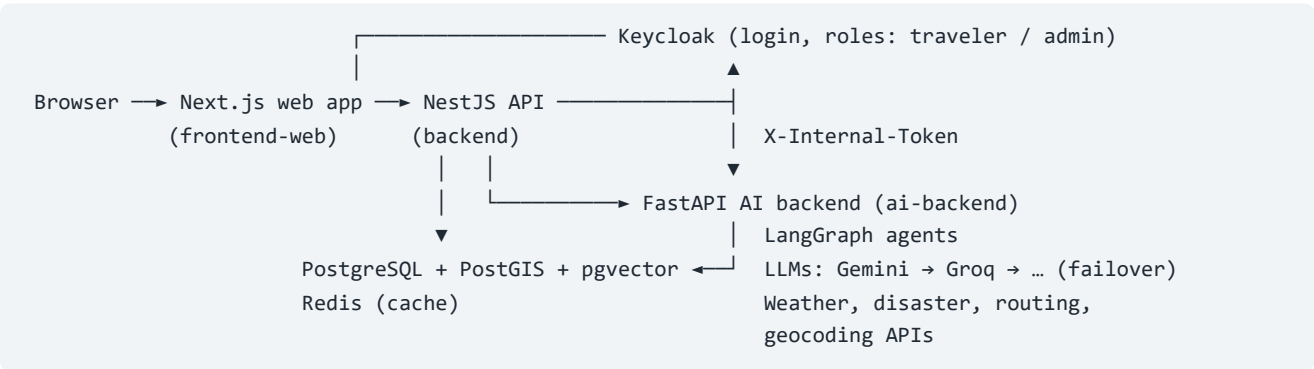
**Group 08.** An AI-powered, multi-agent trip planner for Sri Lanka.

- Live system: <https://aismartjourney.vercel.app>
- Source code: GitHub organisation **SmartJourney-SmartTourismProject**

## 1. What the system does

A traveler describes a trip in plain language. A multi-agent AI backend turns that into a day-by-day itinerary built from **verified, real data**: places, photos, entry fees, events, weather, disaster alerts and travel times. The itinerary stays within the traveler's budget. The traveler can refine the plan by chatting (only the affected day is rebuilt), save it, track expenses against it, and browse places directly. Administrators review imported data before it goes live, manage users, see analytics, and configure the AI models.

## 2. Architecture



| Repository    | Technology                                                                     | Responsibility                                                                                                                                                                         |
|---------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| frontend-web  | Next.js 14 (App Router), TypeScript, Tailwind, Zustand, next-auth, Leaflet map | All screens. Talks <b>only</b> to the NestJS API.                                                                                                                                      |
| backend       | NestJS, Prisma, SQL migrations, Keycloak JWT                                   | The trust boundary: authentication and roles, users, chat, trips, budget, explore, admin. Forwards planning requests to the AI backend with a shared secret. Owns the database schema. |
| ai-backend    | Python, FastAPI, LangGraph                                                     | The planning pipeline, data connectors and scrapers, the RAG knowledge base, the LLM failover chain, and the deterministic fallback planner.                                           |
| Documentation | Markdown and PDF                                                               | SRS, design document, test plan, manuals.                                                                                                                                              |

## 2.1 The planning pipeline (ai-backend)

validate → policy check → slot filling → location → calendar → weather / disaster → recommendation  
agent → planner agent → output validation → respond

- **The LLM understands, the database supplies the facts.** The planner can only use places from the verified database, so it can't invent locations.
- **Output is validated, not trusted.** Plans must match the schema, have coordinates inside Sri Lanka, and stay within budget. A failed plan triggers a repair attempt.
- **Failover.** Models are tried in the admin-configured order (e.g. Gemini, then Groq). If every model fails, a **deterministic rule-based planner** returns a valid, budget-checked plan ( `plan_source = fallback` ), so users never get a server error.
- **Targeted follow-ups.** "Make day 2 cheaper" rebuilds only day 2. The other days stay byte-identical.

## 2.2 Data

- 25 districts and about 6,500 listings, from **OpenStreetMap**, **Booking.com**, **Wikidata** and others, with photos.
- **Events** and **heritage-site entry fees** come from scrapers, and go live only after **admin approval**.
- A **knowledge base** (Wikivoyage and curated notes) with vector embeddings, used for Q&A answers with cited sources.
- The schema is plain SQL in `backend/db/migrations/` (18 files), applied by `backend/db/migrate.py`.

## 2.3 Security

- Keycloak issues RS256 JWTs. NestJS verifies the signature, issuer and audience, then applies role guards ( `@Roles('admin')` ).
- Every query is **scoped by user id**. Another user's resource returns **404**.
- The AI backend accepts planning calls only with the `X-Internal-Token` shared secret, and is not exposed publicly.
- API keys saved in the admin panel are **encrypted** ( `SETTINGS_ENCRYPTION_KEY` ).
- Password policy: 8–12 characters, with upper and lower case, a number and a symbol. Email verification is on.

## 3. Production deployment

| Part                                                           | Where                                                                                                                                                                  |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web app                                                        | <b>Vercel</b> (project <code>aismartjourney</code> , Mumbai region) at <code>https://aismartjourney.vercel.app</code>                                                  |
| NestJS API, AI backend, Keycloak (+ its DB), PostgreSQL, Redis | <b>One AWS EC2 instance</b> (Ubuntu 24.04, Mumbai <code>ap-south-1</code> ) running Docker Compose ( <code>backend/deploy/compose.prod.yml</code> )                    |
| HTTPS                                                          | <b>Caddy</b> reverse proxy with automatic Let's Encrypt certificates. The hostnames are <code>api.&lt;ip&gt;.sslip.io</code> and <code>auth.&lt;ip&gt;.sslip.io</code> |
| Images                                                         | GitHub Container Registry (GHCR), tagged with the commit SHA                                                                                                           |

| Part    | Where                                                                                              |
|---------|----------------------------------------------------------------------------------------------------|
| Backups | backend/deploy/backup.sh (nightly pg_dump of both databases, keeps the last 7), plus EBS snapshots |

Only ports 80 and 443 (and SSH) are open. The database, Redis, the API and the AI backend are reachable only inside the Docker network, through Caddy.

### 3.1 CI/CD (GitHub Actions, .github/workflows/ci.yml in each repo)

| Repo         | Test job (every push and pull request)                   | Deploy job                                                                                       |
|--------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| backend      | npm ci → prisma generate → lint → unit tests → e2e tests | On main : build images → GHCR → SSH to EC2 → deploy.sh (health-checked, with automatic rollback) |
| ai-backend   | pip install → pytest -m "not external"                   | On main : build image → GHCR → deploy as above                                                   |
| frontend-web | npm ci → lint → unit tests                               | On new-main : deploy to Vercel                                                                   |

**Tests gate deployment.** A commit that fails lint or tests never reaches production. Step-by-step server setup: backend/deploy/MANUAL\_SETUP.md .

## 4. Where to find things (submission map)

| Requirement         | Location                                                                                                                                                                                                                                                                                                      |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source code         | ai-backend/app/ , ai-backend/main.py ; backend/src/ , backend/db/ , backend/prisma/ ; frontend-web/src/                                                                                                                                                                                                       |
| Test scripts        | ai-backend/tests/ (pytest), ai-backend/scripts/e2e_check.py , ai-backend/scripts/eval_rag.py ; backend/src/**/*.spec.ts (unit), backend/test/*.e2e-spec.ts ; frontend-web/src/**/*.__tests__/*.spec.ts , frontend-web/e2e/ (Playwright, axe)                                                                  |
| Build scripts       | backend/Dockerfile , ai-backend/Dockerfile , backend/db/Dockerfile , backend/deploy/*.Dockerfile ; package.json scripts, requirements.txt ; backend/docker-compose.yml ; backend/deploy/{deploy.sh, release.sh, compose.sh, backup.sh, restore-data.sh} ; .github/workflows/ci.yml (each repo)                |
| Configuration files | */.env.example , backend/deploy/.env.prod.example , backend/deploy/compose.prod.yml , backend/deploy/Caddyfile , backend/keycloak/realm-export.json , frontend-web/vercel.json , next.config.js , tailwind.config.js , nest-cli.json , tsconfig*.json , pytest.ini , vitest.config*.ts , playwright.config.ts |
| User manuals        | Documentation/User Manual/ (this folder)                                                                                                                                                                                                                                                                      |
| Executable version  | The live site; or a local run with Docker ( 3_Local_Setup_Guide ) using the bundled data ( backend/db/demo-data/restore.sh )                                                                                                                                                                                  |

| Requirement                             | Location                                                           |
|-----------------------------------------|--------------------------------------------------------------------|
| Requirements, design and test documents | Documentation/ (SRS, design document, test plan report, Testing/ ) |

## 5. Known limitations

- The free-tier LLM quotas (Gemini daily, Groq per-minute) limit how often the AI planner runs. The fallback planner covers the gaps.
- Restaurants have no free photo source, so a placeholder is shown.
- The Notifications and Subscription settings are interface previews only.
- Google sign-in needs a Google OAuth client configured for each environment. Email and password sign-in always works.
- The mobile app (Flutter) is planned but not started.